AI Changes the Delivery System and the Product Itself

New capabilities shift constraints. Engineering must adapt around them.





HOW WE BUILD

AI helps a developer write software.

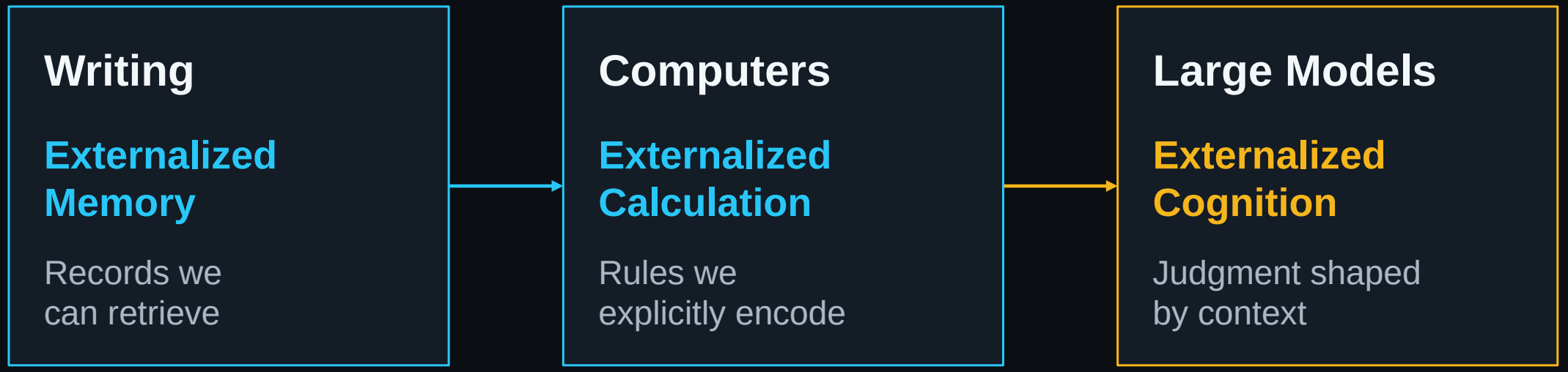
WHAT WE BUILD

AI interprets a user’s request at runtime.

Vitalii Oborskyi Head of Delivery & PMO · Author of Uncertainty Architecture

oborskyivitalii@gmail.com https://www.linkedin.com/in/vitaliioborskyi/

A New Capability Changes the Delegation Boundary

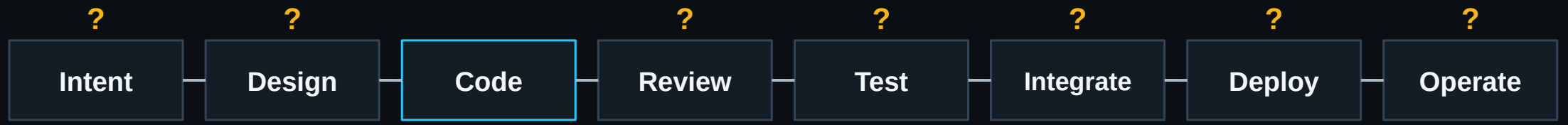


No assumption of will, consciousness or human-like understanding is required.

HOW: rebalance the delivery system.
WHAT: engineer behavior, evidence and control.

We Accelerated a Step. Not the System.

AI can affect every stage. Downstream gains are not automatic.



Illustrative team: changes per week; comparable work, fixed downstream capacity.

	Code	Review	Integrate	BACKLOG
BEFORE Balanced flow	8	8	8	0 no accumulation
AFTER AI Other stages have not sped up	100	8	8	+92 unfinished / week

THEORY OF CONSTRAINTS

Push system: no WIP limit, no discarded work.

Delivery stays at 8/week. WIP grows; overload can reduce quality.

The Evidence Is Already Messy

Different studies measure different outcomes — not one universal productivity effect.

NBER · SEP 2026 · >500K GITHUB DEVS

25.5× changed LOC

3.4× commits

1.3× releases

Cumulative across three tool generations

Changed LOC = additions + deletions.
Fig. 1 rounded; async releases not estimated.

More apps. Low-usage shares rise.

iOS: <10 ratings **≈78% → 87%**

Chrome: <10 downloads **19% → 33%**

4 marketplaces; iOS / Chrome shown.
Jan 2025–Apr 2026; first 3 months.

DORA · 2025 · n = 4,867

Respondents reporting improvement:
Productivity >80% · Code quality 59%

Higher adoption ↔ throughput + instability

Instability: failed production changes + unplanned fixes.

Survey association; no causal estimate.

GITCLEAR · SHARES OF CHANGED LINES

Metric	2021[1]	2026[2]
Moved code	24.65%	3.8% YTD
Copy/paste	8.66%	15.7% H1

[1]2025 report [2]2026 report; different samples.

Snapshots only; no comparable trend or causal estimate.

METR · MAY 2026 · n = 349

1.4–2× self-reported work value

Medians across three questions.

Technical workers; no causal estimate.

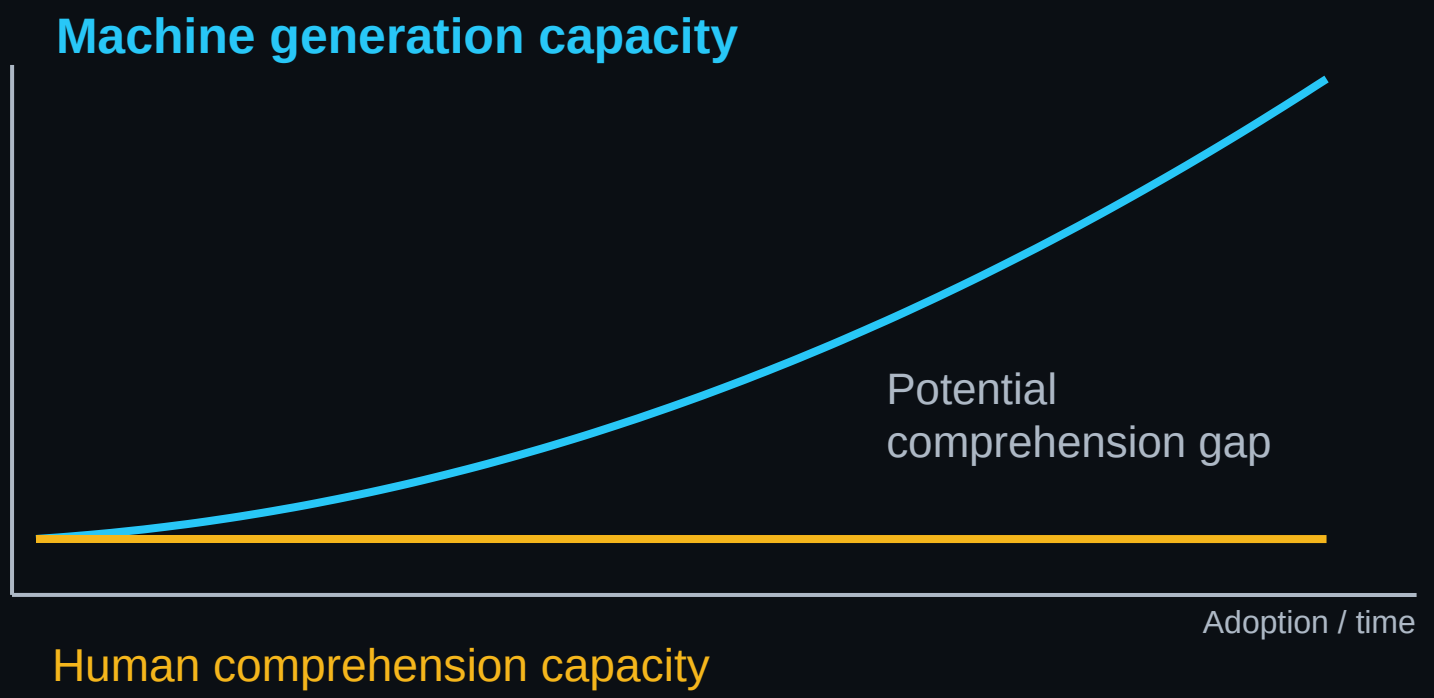
AGARWAL · JAN 2026 · 1,197 REPOS

Cognitive complexity after agent adoption

Agent-first +34.85% · Prior AI-IDE +42.87%

Matched controls; 19,236 repo-months.

The New Scarcity Is Human Comprehension



Can we generate faster?

YES

Can review keep pace?

NOT AT THIS RATE

Will we understand it
in 5 years?

?

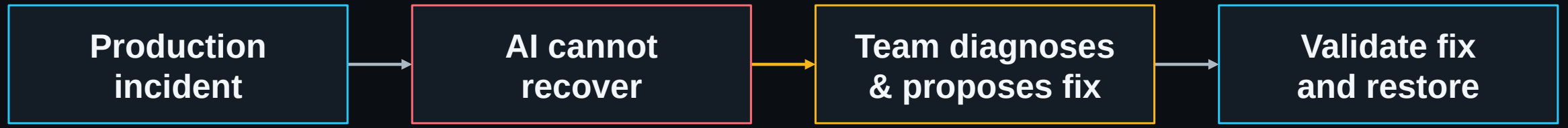
Illustrative overload hypothesis — motivated by slide 4, not measured by those studies.

Generation can scale faster than comprehension.

The Team Is the Last Line of Defense

AI code generators are technical systems with limits.

Their failure to fix an incident belongs in the project risk model.



Contain impact. Roll back or use fallback where safe.

Stop at the agreed retry limit. Preserve traces and changes.

Reconstruct intent, dependencies and system invariants.

Test the repair. Restore under incident owner authority.

Hypothetical scenario. Rehearse the human recovery path before production.

If AI cannot recover the system, the team still must.

We Are Searching for a New SDLC Equilibrium

OLD EQUILIBRIUM	TODAY	NEW EQUILIBRIUM
Human-led flow	Generation can outpace review	Sustainable flow?

WATCH / MEASURE

FLOW & REVIEW
Review age, batch size and WIP
Lead time
CODEBASE & MAINTAINABILITY
Churn, duplication and complexity
Code / dependency growth vs value
DELIVERY & PRODUCTION
Release throughput and stability
Incidents, rollback and recovery time
TEAM, OWNERSHIP & RECOVERY
Review / on-call load and ownership
Team health; recovery without AI

EXPLORE / TEST

FLOW CONTROL
Small batches and WIP limits
Match generation to review capacity
SPEC-FIRST / DISPOSABLE CODE
Trusted specs and tests as anchors
Regenerate only when cheaper
KNOWLEDGE FOR HUMANS + AGENTS
Living docs and ownership maps
Knowledge retrieval for people + agents
AI-ASSISTED REVIEW & RECOVERY
AI review plus static and eval gates
Code + docs review; recovery drills

Candidate countermeasures — not proven fixes.

Test the gap, delivery, quality, full cost and team health.

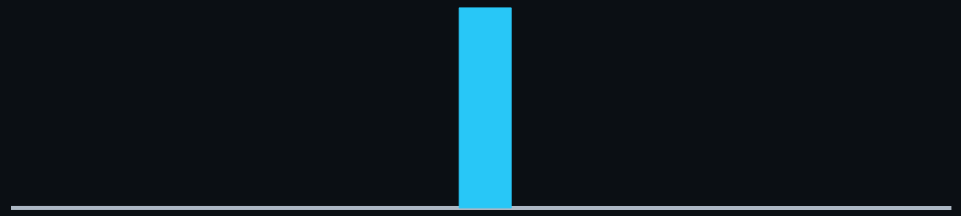
When Software Starts Thinking

PRODUCT / PO · BA · PROJECT MANAGER · ENGINEERING · QA · OPERATIONS

A system where consequential runtime responsibility partly depends on probabilistic Model Judgment.

Software 1.0

$$y = f(x)$$



Same input, state and version: same result.

Software with Model Judgment

$$y \sim P(y \mid x)$$



The same request can produce different results.

Same request, different responses. Context, model and configuration also matter.

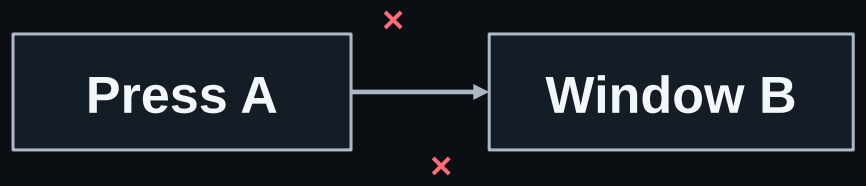
IT assistant: help employees recover access using approved guidance and permitted actions.

Schematic distribution, not measured data. Shape depends on the task.

Requirements Become Boundaries

PRODUCT MANAGER / PRODUCT OWNER · BA · QA / DEV · PROJECT MANAGER

BEFORE: SPECIFY THE PATH



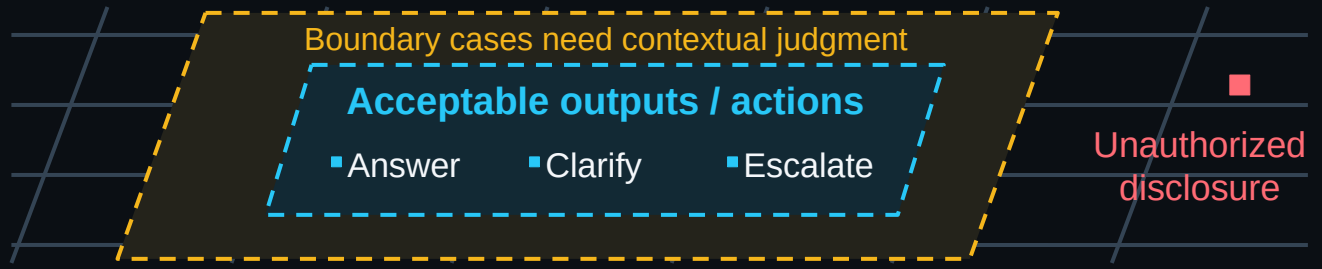
Given the same state,
A must open window B.

Expected states and branches
anchor the acceptance checks.

× Potential violations of the specified path

NOW: SPECIFY A SPACE OF VARIANTS

INPUT: intents, context, data and permissions



Semantic limits are harder to make unambiguous

Schematic space, not a measured probability or distance

MORE COMBINATIONS MAKE SPECIFICATION AND COVERAGE HARDER

Inputs × context × outputs create more ways to miss a requirement.

Known scenarios can pass while unseen combinations fail in production.

ISO/IEC 25059: quality model. ISO/IEC TS 25058: evaluation guidance.

Teams still define semantic criteria, examples, counterexamples and prohibitions.

Product / PO + BA: criteria. QA / Dev: checks. PM: decisions, experts and time.

One Green Test Proves Almost Nothing

QA + DOMAIN EXPERTS · DEVELOPERS · PRODUCT / PO · PROJECT MANAGER

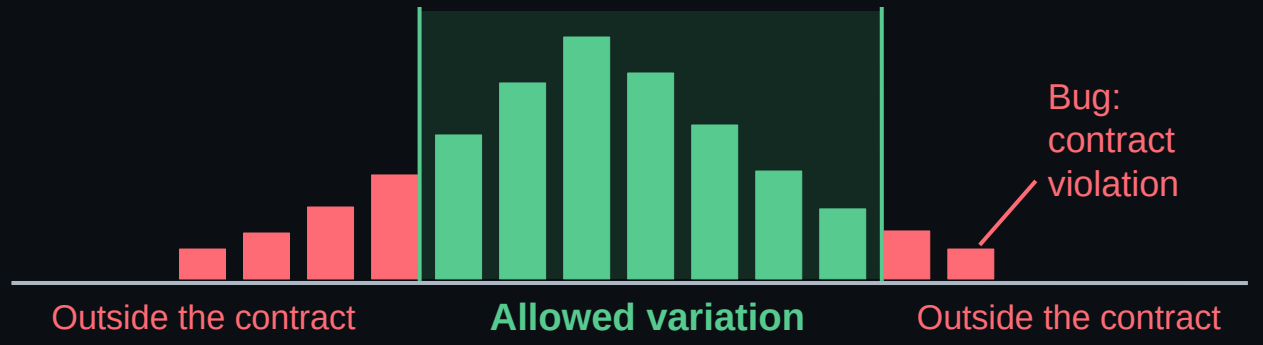
WHAT COUNTS AS A BUG

System behavior violates an approved requirement.

IT assistant: wrong recovery steps or another user's private data.

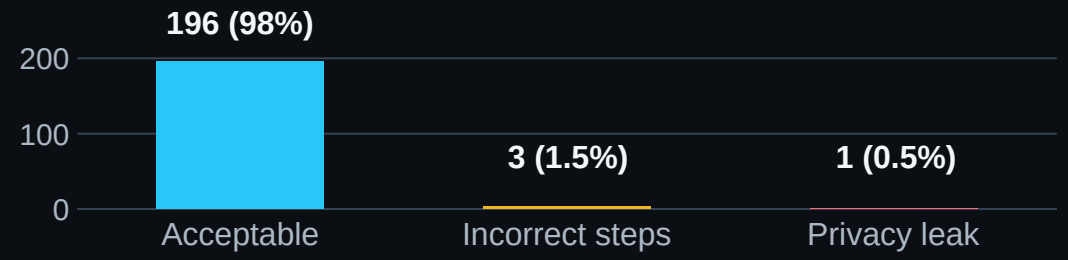
The code may stay unchanged while behavior changes.

APPROVED BUSINESS TOLERANCES



Schematic projection, not measured data. Limits are product-specific.

ILLUSTRATIVE INDEPENDENT SAMPLE · n = 200



4 / 200 = 2% Any contract violation

95% Wilson interval ≈ 0.8%–5.0%

3 incorrect instructions → rework

1 privacy leak → critical breach

Critical breach → BLOCK RELEASE

Golden Set: probe known scenarios. Representative sample: estimate error rates.

QA + domain experts calibrate scoring. Eval Gates compare evidence with release criteria.

A low average error rate does not excuse a prohibited disclosure.

DoR / DoD Become Risk Contracts

PRODUCT / PO + BA · DEVELOPERS + QA · PROJECT MANAGER · OPERATIONS

Thinking Systems add behavioral limits, cost envelopes and statistical evidence to delivery contracts.

CONTRACT	TRADITIONAL DoR / DoD	ADDED FOR THINKING SYSTEMS
DoR READY	Specified behavior + acceptance criteria Dependencies and feasibility Test approach and operating limits	Business tolerances + prohibited outcomes Golden Set, sampling and control / HITL plan Cost, latency and response-capacity budget
DoD DONE	Implementation reviewed Required tests + acceptance checks pass Operational handover complete	Eval Gates meet approved criteria Coverage + uncertainty documented Controls / human fallback tested within budget
RELEASE	Decision against agreed criteria Operational owner accepts the handover	Error bounds + residual risk accepted Monitoring, response and rollback ready Incidents update regression / gates / monitors

DoR is a team practice. DoD is a quality commitment. Release is an authorized decision.

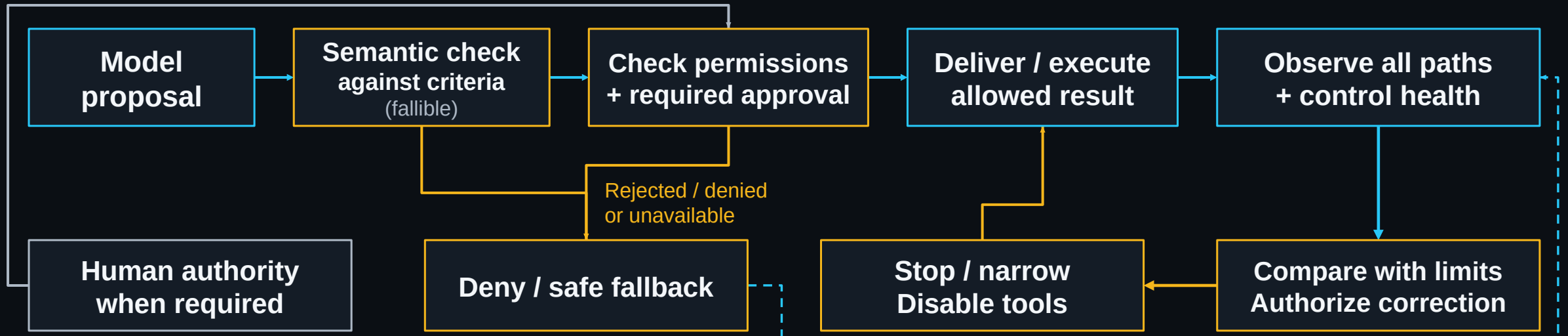
Beyond a green run: measured behavior within approved limits, with a complete, operable control loop.

Production Needs a Control Loop

Validate control effectiveness. Telemetry alone does not establish correctness.

ARCHITECT + DEV: implement · QA: verify · SERVICE OWNER: operate · PM: dependencies

IT ASSISTANT: CONTROL BEFORE ACTION AND AFTER DEPLOYMENT



Evidence, decision rights, competence, time and capacity

Check access before disclosure. Feedback cannot undo an irreversible effect.

LATENCY & PEOPLE

Can control act before consequence?
Is human capacity real?

FULL CONTROL COST

Tokens, evaluations, retries and human review.

EXTERNAL CHANGE

Model, routing and provider assumptions can change.

No reliable, operable control path? Narrow, redesign or reject this AI path.

The Roles Move With the System

PROJECT MANAGER · Product / PO · BA · QA · Dev · Architect · Operations

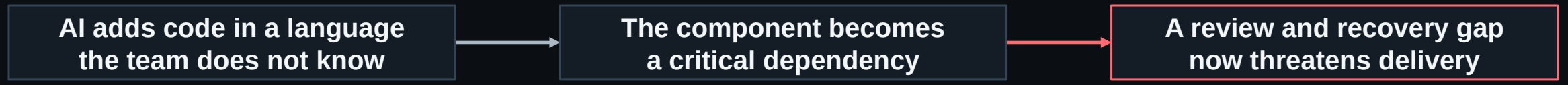
Two shifts change the delivery system, its roles and operating model.

SHIFT	HOW WE BUILD	WHAT WE BUILD
New balance	Faster generation shifts bottlenecks Review, integration and team understanding	Probabilistic behavior enters the product Evaluation, control and incident feedback
New work	Rebalance capacity, review and learning Protect ownership and recovery capability	Assign evaluation and response ownership Plan for model changes and human capacity

PM RESPONSIBILITY: KEEP THE OPERATING MODEL FIT FOR DELIVERY

Track bottlenecks and dependencies. Time-box material unknowns before promising production capability.

ONE EXAMPLE OF A NEW DEPENDENCY: EXPERTISE



Required decisions, evidence and capabilities belong in the delivery dependency plan.

Engineering Rigor Moves — It Doesn’t Disappear

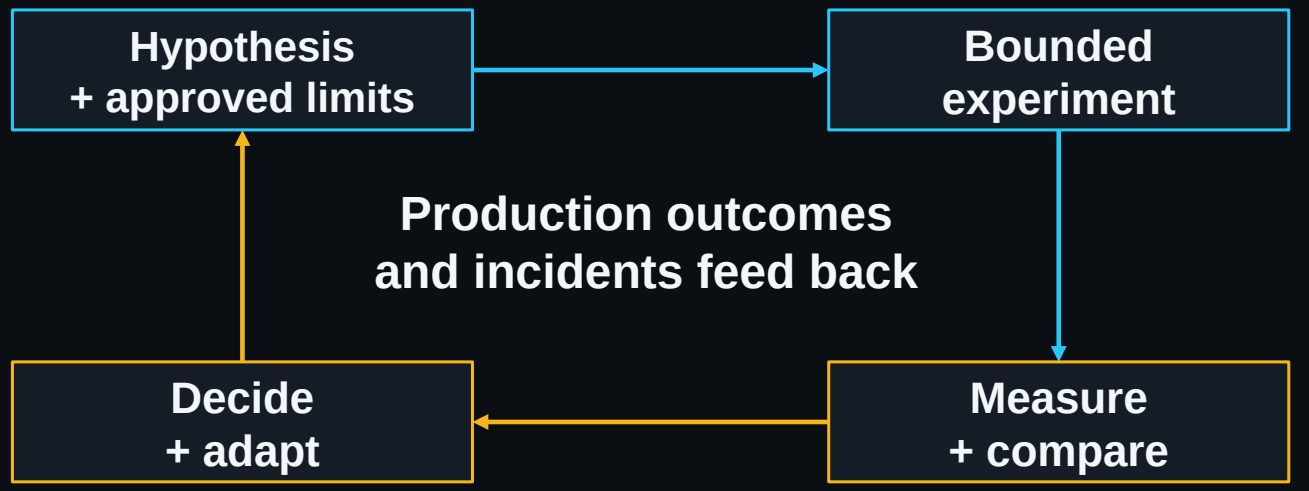
DELIVERY STILL RUNS

Plan → Build → Verify → Ship

Delivery commitments and operational ownership.

THE LABORATORY STAYS OPEN

Re-evaluate after material changes in model, data or authority.



EXISTING TEAM CADENCE · Project Manager coordinates decisions

PLANNING	DAILY WORK	REVIEW EVIDENCE	AUTHORIZED RELEASE	AFTER RELEASE
Product / PO + BA + team Value, limits + evidence	Dev + QA + Architect Build, evaluate, close gaps	Product / PO + team Evidence + uncertainty	Release / risk owner Scope + residual risk	Service owner + responders Monitor, recover + learn

HOW: AI-assisted SDLC Review throughput, rework and team understanding.

WHAT: Thinking Systems Useful outcomes, failure impact and full operating cost.

Start with one workflow: approved limits, evidence, owners and a rehearsed fallback.

More capability still requires engineering responsibility.

Continue the Conversation

HOW WE BUILD

The Subprime Code Crisis

AI-assisted delivery, understanding and long-term ownership

github.com/UncertaintyArchitectureGroup/

The-Subprime-Code-Crisis

WHAT WE BUILD

Uncertainty Architecture

Designing and governing Thinking Systems

github.com/UncertaintyArchitectureGroup/

uncertainty-architecture

Vitalii Oborskyi oborskyivitalii@gmail.com https://www.linkedin.com/in/vitaliioborskyi/